



AZAPA
ENGINEERING

ナレッジ共有型プロジェクト
立ち上げ・運用標準ガイドライン
(**GitHub Base**)

作成日: 2025年12月01日

AZAPAエンジニアリング株式会社

エンジニアリング事業部 先進開発部門 第2セクション

藤平



はじめに.....	4
リポジトリ作成・構成ルール.....	5
リポジトリ構成定義.....	5
セットアップ手順(管理者向け).....	5
Step1. ポータルリポジトリの作成.....	5
Step2. アプリリポジトリの作成.....	6
Step3. 親子連携(Subtree登録).....	7
Step4. MkDocsメニュー更新.....	7
GitHub Project 運用ルール.....	8
Projectの作成設定.....	8
運用ビュー(View)の標準化.....	8
開発フロー(Docs as Code).....	9
ブランチ命名規則.....	9
実装・PR(Pull Request)作成ルール.....	9
・同時修正.....	9
・Issue紐づけ.....	9
・Project設定.....	9
レビュー・承認・マージルール.....	10
承認要件(Branch Protection Rule).....	10
レビュー時のチェックリスト.....	10
マージ戦略.....	10
Git Subtree & CI/CD アーキテクチャ.....	11
運用原則.....	11
自動連携フロー.....	12
1. 子リポジトリ(Application).....	12
2. 親リポジトリ(Portal).....	12
3. 管理者.....	12
手動コマンド(トラブル時のみ).....	13

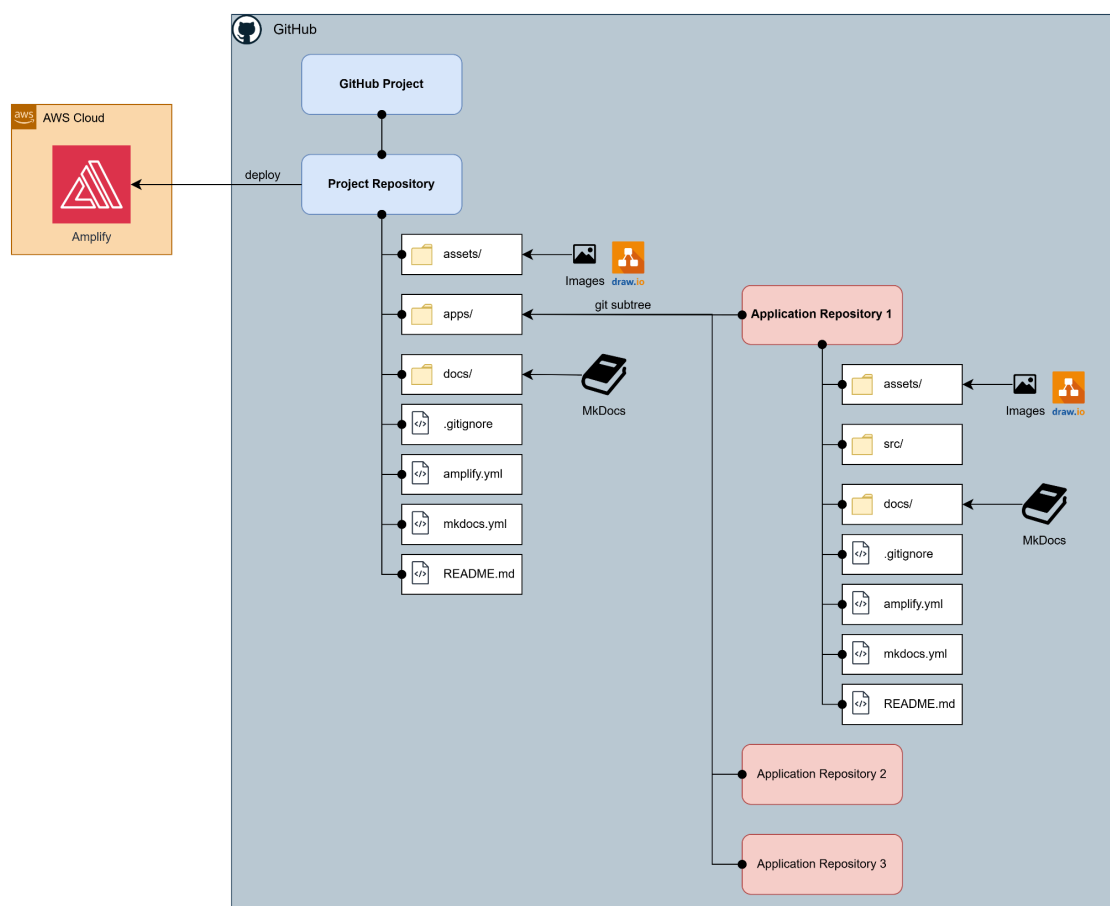
はじめに

本ガイドラインは、プロジェクト立ち上げおよび開発運用における標準ルールを定めたものです。
「コードとドキュメントの一元管理(Docs as Code)を実現し、属人化の排除とナレッジの陳腐化を防ぐことを目的としています。

システム構成概要:

- GitHub Project(案件進捗), GitHub Repositories(ソース/ドキュメント)による管理
 - Git Subtree による親子リポジトリ構造
 - MkDocs + AWS Amplify(または GitHub Pages)によるナレッジ共有
- ※ GitHub Pagesには認証機能がありません。情報の取扱いには十分注意してください。

構成イメージ図:



リポジトリ作成・構成ルール

プロジェクト開始時は、以下のテンプレートを使用して
「親(ポータル)」と「子(アプリ)」のリポジトリを作成してください。

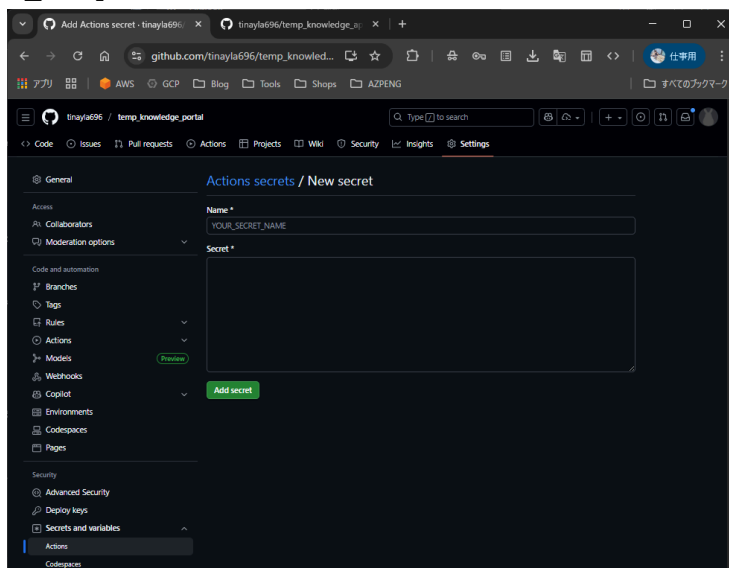
リポジトリ構成定義

種類	命名規則	役割	テンプレート
親リポジトリ Project Repo	<案件名>_portal	ドキュメントポータル、 案件全体管理	temp_knowledge_portal
子リポジトリ Application Repo	<アプリ名>_<言語>	ソースコード、 詳細仕様書	temp_knowledge_app

セットアップ手順(管理者向け)

Step1. ポータルリポジトリの作成

1. Personal Access Token (PAT)を作成します。
2. テンプレート親リポジトリから Private リポジトリを作成します。
3. 作成したリポジトリを開き、[**Setting**] > [**Secrets**] > [**Actions**]を開きます。
4. [**PROJECT_REPO_PAT**]を登録します。(※ GitHub Actions権限不足時のみ)

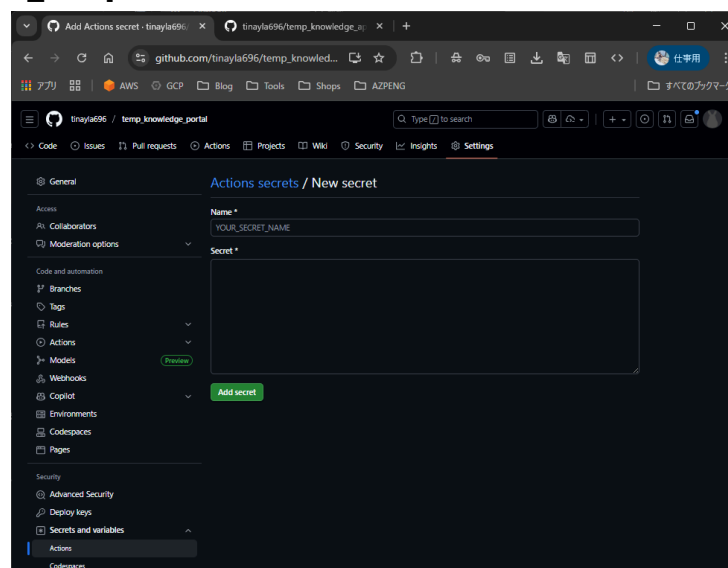




Step2. アプリリポジトリの作成

1. テンプレートリポジトリから Private リポジトリを作成します。
2. GitHub Actionsのワークフローを修正します。
 - a. [.github/workflows/notify_portal.yml] 内の [repository] と [app_name] を修正してコミット。

3. [Setting] > [Secrets] > [Actions] を開きます。
4. [PROJECT_REPO_PAT] を登録します。





Step3. 親子連携 (Subtree登録)

ローカルで親リポジトリに子リポジトリを登録します。

コマンド例:

Shell

親リポジトリでの作業

```
git remote add <アプリ名> <アプリリポジトリURL>
git subtree add --prefix=apps/<アプリ名> <アプリ名> main --squash
git push origin main
```

Step4. MkDocsメニュー更新

親リポジトリの [mkdocs.yml] にアプリへのリンクを追加します。

None

nav:

- Home: index.md
- Applications:
 - 'Sales App': https://<AppのAmplify URL>/



GitHub Project 運用ルール

プロジェクトの進捗管理は **GitHub Projects** で行います。

Projectの作成設定

- 命名: [Project名] Management Board (※ 任意)
- Link: 作成したすべての関連リポジトリ(親・子)で紐づけること。
- 必須カスタムフィールド:
 - [System]: 対象システム名(例: Portal, App-Sales)
 - [Doc Status]: ドキュメント更新ステータス(例: Not Needed, Required, Done)

運用ビュー(View)の標準化

以下の3つのビューを必ず作成してください。

Main Board(看板)

開発のメイン画面になります。([Status] で列を作成)
カードには必ず [Doc Status] を表示させること。

Roadmap(ガントチャート)

[System] でグループ化し、アプリ間のスケジュール干渉を可視化します。

Audit List(テーブル)

[Status: Done] AND [Doc Status: Required] でフィルターします。
これにより、完了したタスクなのに、ドキュメント更新が終わっていないものを監視・撲滅します。



開発フロー (Docs as Code)

「コードとドキュメントはセットである」ことを徹底してください。

ブランチ命名規則

<prefix>/内容 の形式を採用します。

Prefix	用途	例
feature/	新機能追加	feature/login-api
bugfix/	バグ修正	bugfix/feader-layout
docs/	ドキュメントのみ修正	docs/update-manual
hotfix/	緊急修正	hotfix/security-patch
release/	リリース準備	release/ver.1

実装・PR (Pull Request) 作成ルール

・同時修正

コード修正と同じブランチ・同じPRで、[docs/] 配下のドキュメントも修正する。

・Issue紐づけ

PRのDescriptionに [Closes #<Issut番号>] を記述する

・Project設定

PR作成時、右サイドバーのProjectsで該当プロジェクトを選択する。



レビュー・承認・マージルール

品質と履歴の健全性を保つためのルールです。

承認要件 (Branch Protection Rule)

[main] ブランチへのマージには以下を必須とします。

- **Require a pull request:** 直接 *Push* の禁止
- **Require approvals:** 最低1名の承認 (セルフマージの禁止) ※管理者を除く
- **Status checks:** ビルド、テスト、*Lint* の *CI* が成功していること

レビュー時のチェックリスト

レビュアーは承認ボタンを押す前に以下を確認してください。

☐ ドキュメント同期:

機能変更に対し、ドキュメントの修正・追記が含まれているか

ドキュメント修正・追記が完了していない場合、*PR* の差し戻し

☐ プレビューの確認:

Amplify のプレビュー環境で表示崩れがないか

表示崩れ、内容の修正がある場合、*PR* の差し戻し

マージ戦略

Squash and Merge (必須):

特に子リポジトリにおいて、親リポジトリへの取り込み履歴をきれいに保つため、コミットを1つにまとめること



Git Subtree & CI/CD アーキテクチャ

運用原則

- ・単方向同期

親リポジトリは、子リポジトリの変更を **Pull** (取り込み) するのみ とします。

- ・親での直接編集の禁止

apps/ 配下のファイルは、親リポジトリ上で編集することを禁じます。



自動連携フロー

1. 子リポジトリ (Application)

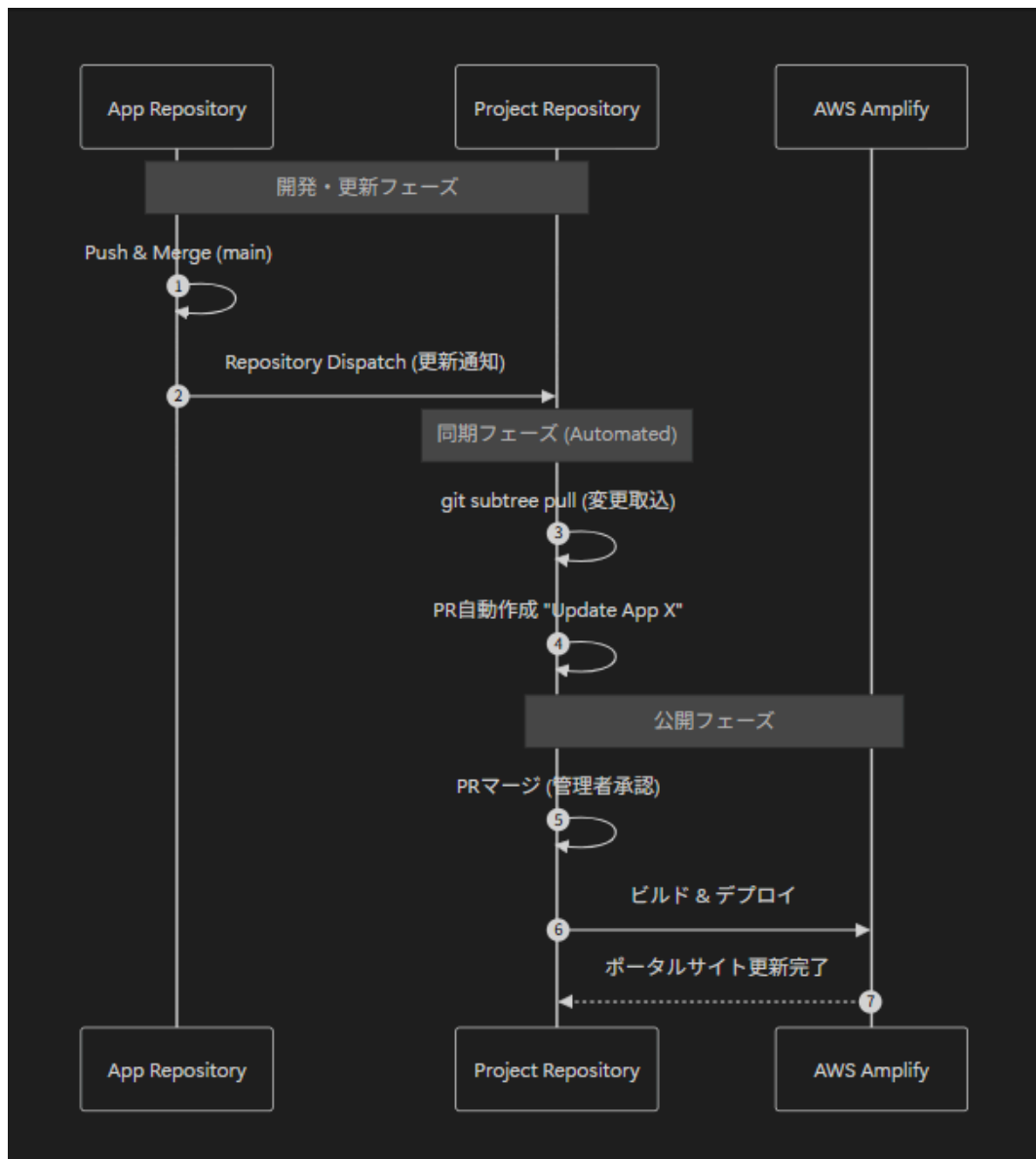
PRマージ >>> Amplify更新 & 親へ更新通知 (Repository Dispatch)

2. 親リポジトリ (Portal)

通知を受信 >>> 自動で [git subtree pull] を実行 >>> 更新取込用のPRを作成

3. 管理者

親リポジトリのPRを確認し、マージしてポータルサイトを更新。





手動コマンド(トラブル時のみ)

自動連携が失敗した場合のみ、実行してください。

Shell

必ず **--squash** をつける

```
git subtree pull --prefix=apps/<アプリ名> <リモート名> main --squash
```